

INTRODUCTION TO PROGRAMMING WITH C++

— Student Handout and Questions —



BASIC
SYNTAX



PROBLEM
SOLVING



CONTROL
STRUCTURES



FUNCTIONS



ARRAYS
AND STRINGS



POINTERS



OBJECT-ORIENTED
PROGRAMMING

SMALL STEPS
BIG IDEAS



*Code
Learn
Create
Grow*

Course Name

Introduction to
Programming with C++

Course Code

CF117

Lecturer

Dr. Shaimaa Ayman

E_mail

Shaimaa.Ayman@iu.edu.eg

CONSOLIDATED EDITION • 10 CHAPTERS • SOLVED PROJECTS

CONTENTS

INTRODUCTION TO PROGRAMMING WITH C++ • CF117

Professional chapter directory / chapters & numbered topics

01-09 Consolidated learning chapters • 10 Worked C++ projects

CHAPTER 01 Programming Foundations.....	6
1.1 Computer systems.....	6
1.2 Programming languages.....	6
1.3 Input-process-output model.....	6
1.4 Plan, code, test, revise.....	6
1.5 Program structure.....	6
1.6 Values and types.....	6
1.7 Variables and arithmetic.....	6
1.8 Style and constants.....	6
1.9 Getting user input.....	6
1.10 Expressions and conversions.....	7
1.11 Overflow and formatting.....	7
1.12 Debug by tracing.....	7
1.13 Key terms in this chapter.....	7
1.14 Guided C++ examples and answer traces.....	7
1.15 Practice and check your understanding.....	8
1.16 Interactive quiz & video lessons.....	9
CHAPTER 02 Decision Making, Loops & Basic Files.....	10
2.1 Conditions and two-way choices.....	10
2.2 Multiple categories.....	10
2.3 Logic, flags, and menus.....	10
2.4 switch and validation.....	10
2.5 The three main loops.....	10
2.6 Counters and accumulators.....	10
2.7 Sentinels and nested loops.....	10
2.8 File storage basics.....	10
2.9 Key terms in this chapter.....	10
2.10 Guided C++ examples and answer traces.....	10
2.11 Practice and check your understanding.....	11
2.12 Interactive quiz & video lessons.....	12
CHAPTER 03 Functions, Arrays & Vectors.....	13
3.1 Divide tasks into functions.....	13
3.2 Inputs and outputs.....	13
3.3 Scope and parameter choices.....	13
3.4 Interfaces and testing.....	13
3.5 Fixed-size collections.....	13
3.6 Traverse and summarize.....	13
3.7 Multiple dimensions and arguments.....	13
3.8 std::vector basics.....	13
3.9 Key terms in this chapter.....	13
3.10 Guided C++ examples and answer traces.....	13
3.11 Practice and check your understanding.....	14
3.12 Interactive quiz & video lessons.....	15
CHAPTER 04 Searching, Sorting & Pointers.....	16
4.1 Linear search.....	16
4.2 Binary search.....	16
4.3 Selection sort.....	16
4.4 Standard algorithms.....	16
4.5 Addresses and dereferencing.....	16

4.6 Pointers and arrays.....	16
4.7 Pointer parameters.....	16
4.8 Ownership and lifetime.....	16
4.9 Key terms in this chapter.....	16
4.10 Guided C++ examples and answer traces.....	16
4.11 Practice and check your understanding.....	17
4.12 Interactive quiz & video lessons.....	18
CHAPTER 05 Strings, Characters & Structured Records.....	19
5.1 Character classification.....	19
5.2 C-strings and termination.....	19
5.3 Conversion and building helpers.....	19
5.4 The std::string class.....	19
5.5 Group related fields.....	19
5.6 Initialization and collections.....	19
5.7 Passing and accessing records.....	19
5.8 Enums and unions.....	19
5.9 Key terms in this chapter.....	19
5.10 Guided C++ examples and answer traces.....	19
5.11 Practice and check your understanding.....	20
5.12 Interactive quiz & video lessons.....	21
CHAPTER 06 File Processing & Class Fundamentals.....	22
6.1 File streams and modes.....	22
6.2 Safe read and write.....	22
6.3 Binary and record data.....	22
6.4 Random access.....	22
6.5 Model data and behavior.....	22
6.6 Encapsulation.....	22
6.7 Constructors and lifetime.....	22
6.8 Key terms in this chapter.....	22
6.9 Guided C++ examples and answer traces.....	22
6.10 Practice and check your understanding.....	23
6.11 Interactive quiz & video lessons.....	24
CHAPTER 07 Object-Oriented Design & Polymorphism.....	25
7.1 Shared versus instance state.....	25
7.2 Copy behavior.....	25
7.3 Operator overloading.....	25
7.4 Inheritance relationships.....	25
7.5 Construction and overrides.....	25
7.6 Virtual dispatch.....	25
7.7 Key terms in this chapter.....	25
7.8 Guided C++ examples and answer traces.....	25
7.9 Practice and check your understanding.....	26
7.10 Interactive quiz & video lessons.....	27
CHAPTER 08 Generic Programming & Linked Lists.....	28
8.1 Exception handling.....	28
8.2 Templates.....	28
8.3 Containers and iterators.....	28
8.4 Nodes and links.....	28
8.5 Traversal and insertion.....	28
8.6 Ownership and variants.....	28
8.7 Key terms in this chapter.....	28
8.8 Guided C++ examples and answer traces.....	28
8.9 Practice and check your understanding.....	29
8.10 Interactive quiz & video lessons.....	29

CHAPTER 09 Stacks, Queues, Recursion & Trees	31
9.1 Stack behavior	31
9.2 Queue behavior	31
9.3 Representations	31
9.4 Self-referential solutions	31
9.5 Proving termination	31
9.6 Classic recursive examples	31
9.7 Tree structure	31
9.8 Binary search trees	31
9.9 Insertion and traversal	31
9.10 Key terms in this chapter	31
9.11 Guided C++ examples and answer traces	31
9.12 Practice and check your understanding	33
9.13 Interactive quiz & video lessons	33
CHAPTER 10 Solved C++ Projects & Answered Examples	35
10.1 Validated Menu Calculator	35
10.2 Student Record Analyzer	35
10.3 Searchable Binary Search Tree	36
10.4 How to submit and test a project	38
10.5 Interactive project quiz & supporting videos	38

Chapter questions: open this PDF from the CF117 Student Hub and click the quiz link at the end of each chapter. The website contains all chapter quizzes. Video lessons require internet.

How to use this handout

This consolidated edition combines the source handout's 20 original topic chapters into nine coherent learning chapters, with an additional tenth chapter containing three complete solved C++ projects. Each original topic has been retained as a compact explanation; repetitive planning pages were removed.

Study route: read the numbered topic → trace the compact worked example → compile the complete C++17 program → try the practice tasks → open the linked self-check quiz. Each original example is a separate program and must not be pasted together with other `main()` functions.

C++17 compilation: `g++ -std=c++17 program.cpp -o program` (run the resulting executable).

Quiz links point to chapter-specific HTML pages on the CF117 Student Hub. Open this book from the published student website; for PDF readers that restrict links, use the quiz shortcuts in the online Book Reader. Quiz results are checked in the student's browser and are not submitted to an LMS.

Source: reorganized from the user-provided CF117 handout (original chapters 1–20); the projects in Chapter 10 are new worked extensions of its sample exercises and Appendix C. Video materials are supplemental third-party resources, not part of the original handout.

CHAPTER 01 | Programming Foundations

Computer systems, C++ syntax, variables, expressions and interactive input.

ORIGINAL HANDOUT COVERAGE • Chapter 1: Introduction to Computers and Programming, Chapter 2: Introduction to C++, Chapter 3: Expressions and Interactivity

LEARNING ROUTE / Read → Trace → Run → Check

1.1 Computer systems

Hardware is the physical system: processor, memory, storage, and input/output devices. Software supplies the instructions that make the hardware perform useful tasks. An operating system manages resources while application software addresses user problems. A program is a sequence of instructions, not merely a screen design.

1.2 Programming languages

Machine instructions are executed by the processor. Higher-level source languages such as C++ let developers express algorithms more readably; a compiler translates C++ source into object code and a linker combines the necessary pieces into an executable. Errors can occur during compilation, linking, or execution.

1.3 Input-process-output model

A well-defined task identifies what is provided, what transformation is required, and what result must be produced. For rectangle area, inputs are width and height, the calculation is width × height, and the output is an area with an informative label. This model also helps identify invalid input before coding.

1.4 Plan, code, test, revise

First understand the specification and write pseudocode or a flowchart. Next implement and compile the program. Test representative and boundary values, compare actual output with expected output, and revise. Procedural programming organizes steps into functions; object-oriented programming also models related data and behavior as objects.

WATCH OUT A program may compile successfully and still calculate an incorrect answer.

1.5 Program structure

A console application usually begins at `int main()`. A header such as `<iostream>` provides declarations for standard input/output streams. Braces enclose the function body, statements normally end in semicolons, and `return 0` conventionally reports successful completion. The `std::` prefix identifies names from the standard namespace.

1.6 Values and types

`int` represents whole-number values, `double` supports fractional quantities, `char` represents one character, `bool` stores true or false, and `std::string` from `<string>` stores text. Literals are values written directly in source code. Choose names such as `totalCost` rather than single-letter names when the role is not obvious.

1.7 Variables and arithmetic

A variable should be initialized before it is used. Assignment with `=` changes its stored value; `==` will later be used for equality testing. Arithmetic uses `+`, `-`, `*`, `/` and integral remainder `%`. `int / int` uses integer division, so `7 / 2` evaluates to 3. Parentheses make intended calculation order explicit.

1.8 Style and constants

Use `const` for named values that should remain unchanged. Comments beginning `//` explain a reason or intent; `/* ... */` creates a block comment. Consistent indentation and sensible line lengths make the source easier to review and debug. A variable defined inside a block is scoped to that block.

WATCH OUT Never use an uninitialized local variable in a calculation.

1.9 Getting user input

`std::cin >> value` extracts typed input and generally stops at whitespace. Check the stream before using the value. `std::getline` reads a complete line, including spaces; mixing it with `>>` requires handling any pending newline. Prompts should make both required input and expected units clear.

1.10 Expressions and conversions

An expression combines operands and operators into a result. Mixed arithmetic may convert integers into floating-point values. Explicit `static_cast<double>(sum) / count` documents an intended floating calculation. Never divide by zero. Prefer a result type that retains the precision required by the task.

1.11 Overflow and formatting

Integer types have finite ranges; arithmetic that exceeds a signed integer range can cause undefined behavior. Floating-point representations have limited precision. Include `<iomanip>` and use `std::fixed` with `std::setprecision(2)` to display two digits after the decimal point. Display formatting does not change the stored value.

1.12 Debug by tracing

A hand trace records variable values after each statement. A trace for a receipt might have columns for quantity, unitPrice, subtotal, and total. Library operations from `<cmath>` can compute roots and powers, but their input domain and numeric types must still be considered.

WATCH OUT Input extraction can fail; do not continue to use values from a failed extraction.

1.13 Key terms in this chapter

program; hardware; software; source code; algorithm; compiler; input; output • `#include`; `main`; `cout`; identifier; literal; `int`; `double`; `char`; `bool`; `string`; `const` • `cin`; stream extraction; expression; `cast`; overflow; `setw`; `setprecision`; `getline`

1.14 Guided C++ examples and answer traces

MINI-EXAMPLE (original Chapter 1)

WORKED MINI-EXAMPLE | Pseudocode: rectangle perimeter

```
READ width, height
IF either is negative: REPORT invalid
ELSE perimeter = 2 × (width + height)
DISPLAY perimeter
```

How it works / expected result: For width 4 and height 3.5, perimeter is 15. Inputs represent lengths, so negative dimensions must be rejected.

Original Chapter 1: Rectangle area: first complete C++ program

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain `main()`.

```
#include <iostream>
int main() {
    double width = 4.0;
    double height = 3.5;
    double area = width * height;
    std::cout << "Rectangle area = " << area << '\n';
    return 0;
}
```

MINI-EXAMPLE (original Chapter 2)

WORKED MINI-EXAMPLE | Small example: labeled output

```
int pens = 3;
std::cout << "Pens: " << pens << "\n";
```

How it works / expected result: Output is Pens: 3. The quotation marks create a string literal; pens without quotes is a variable.

Original Chapter 2: Type choices and labeled output

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain `main()`.

```
#include <iostream>
#include <string>
int main() {
    const double TAX = 0.14;
    std::string item = "Notebook";
    double price = 50.0;
    double total = price * (1.0 + TAX);
    std::cout << item << " costs " << total << " EGP\n";
    return 0;
}
```

MINI-EXAMPLE (original Chapter 3)

WORKED MINI-EXAMPLE | Small example: avoid integer division

```
int total = 7;
int count = 2;
double mean = static_cast<double>(total) / count;
```

How it works / expected result: The answer is 3.5 rather than 3. Validate `count > 0` before division.

Original Chapter 3: Interactive order total with formatted output

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```
#include <iostream>
#include <iomanip>
int main() {
    int quantity = 0;
    double unitPrice = 0.0;
    std::cout << "Quantity and unit price: ";
    if (!(std::cin >> quantity >> unitPrice) || quantity < 0
        || unitPrice < 0.0) return 1;
    const double total = quantity * unitPrice;
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Total = " << total << " EGP\n";
    return 0;
}
```

INPUT / OUTPUT REFERENCE • Original Chapter 1

Sample input / starting data: width = 4.0, height = 3.5

Expected output: Rectangle area = 14

ANSWER TRACE • Original Chapter 1

1. Identify the two input values already assigned to variables.
2. Multiply width by height to calculate the area.
3. Send a descriptive label and the numerical result to the output stream.
4. Replace width with 0 and predict the output before compiling.

INPUT / OUTPUT REFERENCE • Original Chapter 2

Sample input / starting data: item = Notebook; price = 50; tax = 0.14

Expected output: Notebook costs 57 EGP

ANSWER TRACE • Original Chapter 2

1. Read each declaration and identify whether it is a variable or constant.
2. Compute 50×1.14 by hand.
3. Check how successive << operators combine text with values.
4. Change the item name and tax rate and predict both effects.

INPUT / OUTPUT REFERENCE • Original Chapter 3

Sample input / starting data: 3 12.5

Program output (for the sample input above): Quantity and unit price: Total = 37.50 EGP

ANSWER TRACE • Original Chapter 3

1. Start quantity at 0 and unitPrice at 0.0.
2. Extract 3 and 12.5; verify the input succeeds.
3. Compute the product and note its type.
4. Apply the display format only when printing the total.

1.15 Practice and check your understanding

01. [Original Ch 1] Distinguish a compiler error from an algorithmic error.
02. [Original Ch 1] Draw an IPO diagram for converting hours into minutes.
03. [Original Ch 1] Write pseudocode for finding the larger of two numbers.
04. [Original Ch 1] Explain the role of the processor, memory, and storage.
05. [Original Ch 1] EXTENSION: Create an algorithm and C++ program that calculates the perimeter of a rectangle, then test a square and a zero-width case.
06. [Original Ch 2] Write the shortest valid Hello World program with standard output.
07. [Original Ch 2] Identify invalid identifiers: 2total, grossPay, first-name.
08. [Original Ch 2] Give declarations for age, GPA, initial, and full name.
09. [Original Ch 2] Explain why const is appropriate for a fixed tax rate.
10. [Original Ch 2] EXTENSION: Build a simple shopping receipt that prints a product name, quantity, unit price, subtotal, and total.
11. [Original Ch 3] Write a program that reads two floating values and displays their average.
12. [Original Ch 3] Explain why $5 / 2$ differs from $5.0 / 2$.

13. [Original Ch 3] Compare int, double, and long long for large quantities.
14. [Original Ch 3] Read a full name using getline after one numeric input.
15. [Original Ch 3] EXTENSION: Build a length converter that reads meters, calculates centimeters and millimeters, and prints labeled results with suitable formatting.

1.16 Interactive quiz & video lessons

Interactive quiz — 18 auto-marked questions, instant feedback, explanations and chapter score.

[OPEN CHAPTER QUIZ ↗](#) • Opens this chapter's quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • Verified public video lessons (external / internet needed)

[C++ foundations: introduction, setup, variables and types \(freeCodeCamp\) ↗](#)

[Interactive input and calculator worked example \(freeCodeCamp, 59:41\) ↗](#)

 [BACK TO CONTENTS](#)

CHAPTER 02 | Decision Making, Loops & Basic Files

Conditional logic, repetition, validation and an introduction to text files.

ORIGINAL HANDOUT COVERAGE • Chapter 4: Making Decisions, Chapter 5: Loops and Files

LEARNING ROUTE / Read → Trace → Run → Check

2.1 Conditions and two-way choices

Comparisons use `==`, `!=`, `<`, `<=`, `>`, and `>=`. An `if` executes its body when its condition is true. An `if/else` guarantees that exactly one of two branches is selected. Put braces around branch bodies, including one-line bodies, to keep later edits safe.

2.2 Multiple categories

An `else-if` chain evaluates conditions from the top and executes only the first successful branch. Validate impossible values first. When ranges overlap, test the most specific or highest-priority category first; otherwise a broader condition can hide a later branch.

2.3 Logic, flags, and menus

Boolean flags can record state such as `hasTicket`. Logical `&&` requires two conditions; `||` requires one; `!` negates a Boolean value. Both `&&` and `||` short-circuit. A menu prompts for a choice, evaluates it, and handles unsupported selections explicitly.

2.4 switch and validation

`switch` is suitable for a discrete integral or enum expression, with constant case labels, optional default, and usually `break` to avoid fall-through. Validate stream input before the `switch`. The conditional `?:` operator can express a short two-way value selection, but readability matters more than brevity.

WATCH OUT In C++, `0 <= x <= 100` does not correctly test a numeric range; use `x >= 0 && x <= 100`.

2.5 The three main loops

`while` checks the condition before its body, `do-while` checks afterward and executes at least once, and `for` expresses initialization, condition, and update together. A counter often begins at zero; failure to change the loop state can result in an infinite loop.

2.6 Counters and accumulators

A counter tracks processed items, while an accumulator maintains a sum or similar aggregate. Initialize a running total before the loop and include each valid item once. When calculating a mean, divide only if the count is positive and preserve floating-point precision.

2.7 Sentinels and nested loops

A sentinel is an input value outside the normal data domain that ends repetition. For a class of marks from 0–100, -1 is a possible sentinel. A nested loop can print a grid or examine row/column pairs. Carefully identify whether `break` exits the intended loop.

2.8 File storage basics

`<fstream>` provides `std::ifstream` for input and `std::ofstream` for output. Confirm that a file opens successfully before processing. Reading with `while (input >> value)` uses the stream state as the loop condition. Files are persistent data sources and not a substitute for input validation.

WATCH OUT Treating a sentinel as ordinary data changes the final total and average.

2.9 Key terms in this chapter

`if`; `else`; `else if`; `bool`; `&&`; `||`; `!`; `switch`; `case`; `default` • `loop`; `while`; `for`; `do-while`; `accumulator`; `sentinel`; `ifstream`; `ofstream`

2.10 Guided C++ examples and answer traces

MINI-EXAMPLE (original Chapter 4)

WORKED MINI-EXAMPLE | Small example: correct range condition

```
int mark = 50;
bool valid = (mark >= 0 && mark <= 100);
```

How it works / expected result: `valid` is true. C++ does not support the mathematical shorthand `0 <= mark <= 100`.

Original Chapter 4: Validated score classification

The following is a complete, original C++17 example. Compile it as its own `.cpp` file; do not combine several examples that each contain `main()`.

```
#include <iostream>
int main() {
    int score = 0;
    std::cout << "Score (0-100): ";
    if (!(std::cin >> score)) return 1;
    if (score < 0 || score > 100) {
        std::cout << "Invalid score\n";
    } else if (score >= 85) {
        std::cout << "Excellent\n";
    } else if (score >= 50) {
        std::cout << "Pass\n";
    } else {
        std::cout << "Fail\n";
    }
    return 0;
}
```

MINI-EXAMPLE (original Chapter 5)

WORKED MINI-EXAMPLE | Small example: a counted loop

```
int sum = 0;
for (int n = 1; n <= 3; ++n) sum += n;
std::cout << sum << "\n";
```

How it works / expected result: Output is 6. State the initial value, termination condition and update before coding any loop.

Original Chapter 5: Sentinel-driven summary

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```
#include <iostream>
int main() {
    int value = 0, count = 0, sum = 0;
    std::cout << "Enter positive values (-1 ends): ";
    while (std::cin >> value && value != -1) {
        if (value >= 0) {
            sum += value;
            ++count;
        }
    }
    if (count > 0)
        std::cout << "Average = " << sum / double(count) << '\n';
    else
        std::cout << "No values provided\n";
    return 0;
}
```

INPUT / OUTPUT REFERENCE • Original Chapter 4

Sample input / starting data: 85

Program output (for the sample input above): Score (0-100): Excellent

ANSWER TRACE • Original Chapter 4

1. Read the score and reject nonnumeric input immediately.
2. Check the valid domain before evaluating categories.
3. The first matching valid category handles 85, so later conditions are skipped.
4. Try -1, 49, 50, 85 and 101 as boundary cases.

INPUT / OUTPUT REFERENCE • Original Chapter 5

Sample input / starting data: 2 4 6 -1

Program output (for the sample input above): Enter positive values (-1 ends): Average = 4

ANSWER TRACE • Original Chapter 5

1. Count begins at zero and sum begins at zero.
2. Three nonnegative inputs change count to 3 and sum to 12.
3. -1 stops the while condition and is not included.
4. Compute 12 / 3.0, then test the case where the first input is -1.

2.11 Practice and check your understanding

01. [Original Ch 4] Draw a decision tree for classifying a temperature.
02. [Original Ch 4] Write an even/odd checker using if and remainder.
03. [Original Ch 4] Compare three numbers and print the largest.
04. [Original Ch 4] Write a four-option arithmetic menu with switch.
05. [Original Ch 4] EXTENSION: Create a menu-based ticket calculator that validates age and explains the price category.

06. [Original Ch 5] Convert a while loop into a for loop with the same result.
07. [Original Ch 5] Write a validated positive-number prompt.
08. [Original Ch 5] Compute the sum from 1 to n.
09. [Original Ch 5] Use nested loops to print a multiplication grid.
10. [Original Ch 5] EXTENSION: Create a small attendance counter that repeatedly reads 0 or 1, stops on -1, and prints present, absent, and total counts.

2.12 Interactive quiz & video lessons

Interactive quiz — 12 auto-marked questions, instant feedback, explanations and chapter score.

[OPEN CHAPTER QUIZ ↗](#) • Opens this chapter's quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • **Verified public video lessons (external / internet needed)**

[Decisions and if/else \(freeCodeCamp, 1:35:22\) ↗](#)

[Switch and loops \(freeCodeCamp, 2:02:20\) ↗](#)

[🏠 BACK TO CONTENTS](#)

CHAPTER 03 | Functions, Arrays & Vectors

Modular solutions, parameters, array bounds and sequential data processing.

ORIGINAL HANDOUT COVERAGE • Chapter 6: Functions, Chapter 7: Arrays

LEARNING ROUTE / Read → Trace → Run → Check

3.1 Divide tasks into functions

A function represents a meaningful operation such as `computeArea` or `displayMenu`. Its declaration states an interface, its definition provides an implementation, and a call transfers execution to it. The compiler must see a declaration before an earlier call when the definition comes later.

3.2 Inputs and outputs

Arguments provide the function with data. A value parameter receives a copy; changing the copy does not change the caller variable. A value-returning function uses `return` to send a result back; `void` indicates no return value. Use `bool` returns for simple yes/no predicates.

3.3 Scope and parameter choices

Local variables are confined to their block, while global state is broadly accessible and more difficult to reason about. Static local variables retain their values between calls. Reference parameters can modify a caller object and should be used deliberately; `const` references are useful for read-only objects.

3.4 Interfaces and testing

Default arguments simplify optional inputs and overloads allow the same function name with different parameter lists. During development, a stub can provide a placeholder implementation and a driver can test a function without the entire application. Build and test one function at a time.

WATCH OUT A non-void function must return an appropriate value on every executed path.

3.5 Fixed-size collections

An array is a sequence of elements with the same type. C-style array indexing begins at zero, so an array of length five has valid indices 0 through 4. A raw array does not automatically check subscripts. Always derive a loop bound from the actual capacity of the array.

3.6 Traverse and summarize

For loops work well when the index is needed. A range-based for loop reads each element directly and avoids many indexing errors. For sum or average calculations, initialize an accumulator and divide by the number of values only after ensuring it is not zero.

3.7 Multiple dimensions and arguments

Two-dimensional arrays arrange data by row and column. Parallel arrays relate values by sharing an index, but their elements must stay aligned. When passing a raw array to a function, also provide its length because array size information is not generally retained by the parameter.

3.8 `std::vector` basics

The standard vector container manages a dynamic sequence of values. `push_back` adds elements, `size` reports the current count, and `at` provides bounds checking. Unlike a raw array, vector can grow; however, empty vectors still require special handling before accessing their first element.

WATCH OUT Using `scores[5]` for a five-element array is out of bounds; the final valid index is 4.

3.9 Key terms in this chapter

function; prototype; parameter; argument; return; scope; overload; reference • array; index; element; size; bounds; traversal; parallel arrays; vector

3.10 Guided C++ examples and answer traces

MINI-EXAMPLE (original Chapter 6)

WORKED MINI-EXAMPLE | Small example: separate calculation

```
int square(int x) { return x * x; }
// In main: std::cout << square(6);
```

How it works / expected result: Output is 36. A calculation function can be tested without mixing in keyboard input.

Original Chapter 6: Functions for a validated rectangle

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```
#include <iostream>
double rectangleArea(double width, double height) {
    return width * height;
}
int main() {
    double w = 0.0, h = 0.0;
    std::cout << "Width and height: ";
    if (!(std::cin >> w >> h) || w < 0 || h < 0)
        return 1;
    std::cout << "Area = " << rectangleArea(w, h) << '\n';
    return 0;
}
```

MINI-EXAMPLE (original Chapter 7)

WORKED MINI-EXAMPLE | Small example: array index boundaries

```
int values[3] = {4, 8, 12};
std::cout << values[0] << " " << values[2];
```

How it works / expected result: Output is 4 12. Accessing values[3] is out of bounds and must never be attempted.

Original Chapter 7: Count passing scores in an array

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```
#include <iostream>
int main() {
    int scores[5] = {45, 65, 85, 30, 90};
    int passed = 0;
    for (int score : scores) {
        if (score >= 50) ++passed;
    }
    std::cout << "Passed = " << passed << '\n';
    std::cout << "Total = " << 5 << '\n';
    return 0;
}
```

INPUT / OUTPUT REFERENCE • Original Chapter 6

Sample input / starting data: 4 2.5

Program output (for the sample input above): Width and height: Area = 10

ANSWER TRACE • Original Chapter 6

1. main reads and validates data rather than printing from the calculation function.
2. The call rectangleArea(w, h) copies two double arguments.
3. The function returns their product to the output expression.
4. Test the calculation independently using 0, 1, and fractional dimensions.

INPUT / OUTPUT REFERENCE • Original Chapter 7

Sample input / starting data: 45, 65, 85, 30, 90

Expected output: Passed = 3 | Total = 5

ANSWER TRACE • Original Chapter 7

1. The range-based loop visits five values in order.
2. The passing condition succeeds for 65, 85 and 90.
3. Increment passed only for matching values.
4. Change the threshold to 70 and recompute before running.

3.11 Practice and check your understanding

01. [Original Ch 6] Declare and define int square(int x).
02. [Original Ch 6] Implement bool isEven(int x) and test it.
03. [Original Ch 6] Explain the difference between parameters and arguments.
04. [Original Ch 6] Show a function prototype placed before main.
05. [Original Ch 6] EXTENSION: Develop a menu program with separate showMenu(), readChoice(), and executeChoice() functions.
06. [Original Ch 7] Draw a five-element array and label all valid indices.

- 07. [Original Ch 7] Find minimum and maximum in an array.
- 08. [Original Ch 7] Compute an average using a range-based for loop.
- 09. [Original Ch 7] Pass an array and its size to a function.
- 10. [Original Ch 7] EXTENSION: Develop a grade analyzer that prints minimum, maximum, mean, and the count above a chosen threshold.

3.12 Interactive quiz & video lessons

Interactive quiz — 12 auto-marked questions, instant feedback, explanations and chapter score.

[OPEN CHAPTER QUIZ ↗](#) • Opens this chapter's quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • **Verified public video lessons (external / internet needed)**

[Functions and return values \(freeCodeCamp, 1:20:03\) ↗](#)

[Arrays and nested loops \(freeCodeCamp, 1:13:45 / 2:45:21\) ↗](#)

 [BACK TO CONTENTS](#)

CHAPTER 04 | Searching, Sorting & Pointers

Find and arrange values; reason about addresses, references and ownership.

ORIGINAL HANDOUT COVERAGE • Chapter 8: Searching and Sorting Arrays, Chapter 9: Pointers

LEARNING ROUTE / Read → Trace → Run → Check

4.1 Linear search

A linear search scans elements sequentially until the target is found or the collection is exhausted. It works on unsorted input, but a large input may require inspection of every element. Returning -1 from a search over nonnegative indices is one convention for not found.

4.2 Binary search

Binary search repeatedly compares the target with the midpoint of an ordered range. It requires a sorted collection using the same comparison order as the algorithm. After a mismatch, only one half of the range can contain the target, so the candidate interval shrinks rapidly.

4.3 Selection sort

Selection sort repeatedly finds the smallest value in the remaining unsorted suffix and swaps it into the next position. This is easy to trace but compares many pairs on large inputs. Different sorting algorithms trade simplicity, extra memory, and execution time.

4.4 Standard algorithms

Include `<algorithm>` for `std::sort` and `std::binary_search`. On vector data, iterators identify the range. Sort before `binary_search` unless the vector is already ordered. To obtain a matching position, `std::lower_bound` or a hand-written search can be used carefully.

WATCH OUT Do not apply binary search to an unsorted collection and assume the result will be meaningful.

4.5 Addresses and dereferencing

A pointer stores the address of an object. `&value` obtains an address and `*pointer` accesses the object stored at that address. A pointer has a type describing the pointed-to object. Initialize pointers before use; a null pointer deliberately refers to no object.

4.6 Pointers and arrays

For many expressions, an array name converts to a pointer to its first element. Pointer arithmetic moves in units of the pointed-to type, not bytes; it must stay within a valid array range or one-past position under the language rules. Do not dereference a one-past pointer.

4.7 Pointer parameters

A function can receive a pointer and modify the pointed-to object when the pointer is valid and the function permits modification. Nullable pointer parameters require explicit null checks before dereferencing. This is different in syntax from a C++ reference parameter, which must bind to an object.

4.8 Ownership and lifetime

Dynamically allocated memory requires a clear lifetime and an owner. A raw pointer returned by new traditionally requires a matching delete, but modern code can favor smart pointers such as `std::unique_ptr`. A smart pointer releases its owned resource when its lifetime ends, helping prevent leaks.

WATCH OUT Dereferencing `nullptr`, a dangling pointer, or uninitialized pointer invokes undefined behavior.

4.9 Key terms in this chapter

linear search; binary search; sorted order; selection sort; index; `std::sort` • address; `&`; pointer; `*`; `nullptr`; new; delete; `unique_ptr`; dereference

4.10 Guided C++ examples and answer traces

MINI-EXAMPLE (original Chapter 8)

WORKED MINI-EXAMPLE | Small example: why the precondition matters

```
std::vector<int> a = {8, 2, 5};
std::sort(a.begin(), a.end());
bool exists = std::binary_search(a.begin(), a.end(), 5);
```

How it works / expected result: exists is true; sorting first is required unless the input is already ordered under the same comparison.

Original Chapter 8: Sort and search a vector

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> data = {9, 3, 7, 1, 5};
    std::sort(data.begin(), data.end());
    int target = 7;
    bool found = std::binary_search(data.begin(),
                                    data.end(), target);
    std::cout << std::boolalpha << "Found = " << found << '\n';
    std::cout << "Ordered values:";
    for (int value : data) std::cout << ' ' << value;
    std::cout << '\n';
    return 0;
}
```

MINI-EXAMPLE (original Chapter 9)

WORKED MINI-EXAMPLE | Small example: pointer vs value

```
int number = 5;
int* ptr = &number;
*ptr = 8;
```

How it works / expected result: number becomes 8. ptr must refer to a live object before dereferencing.

Original Chapter 9: A checked pointer and unique ownership

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```
#include <iostream>
#include <memory>

int main() {
    auto number = std::make_unique<int>(12);
    int* observer = number.get();
    if (observer != nullptr) {
        *observer += 3;
        std::cout << "Value = " << *number << '\n';
    }
    return 0;
}
```

INPUT / OUTPUT REFERENCE • Original Chapter 8

Sample input / starting data: 9, 3, 7, 1, 5; target = 7

Expected output: Found = true

Ordered values: 1 3 5 7 9

ANSWER TRACE • Original Chapter 8

1. std::sort changes the vector to ascending order.
2. binary_search checks existence, not the index.
3. The output first prints "Ordered values:"; the range-based loop appends the numbers with leading spaces.
4. Try target 4 and predict the Boolean result.

INPUT / OUTPUT REFERENCE • Original Chapter 9

Sample input / starting data: an allocated integer initialized to 12

Expected output: Value = 15

ANSWER TRACE • Original Chapter 9

1. make_unique creates an owned integer.
2. get obtains a non-owning raw pointer; ownership stays with number.
3. The pointer is checked before dereferencing.
4. When main finishes, the unique_ptr releases the owned integer.

4.11 Practice and check your understanding

01. [Original Ch 8] Trace linear search for target 8 in {4, 8, 2, 8}.
02. [Original Ch 8] Explain the sorted-input condition for binary search.
03. [Original Ch 8] Trace selection sort on {5, 1, 4, 2}.
04. [Original Ch 8] Count linear-search comparisons in a worst-case example.

05. [Original Ch 8] EXTENSION: Build a menu that lets the user load scores, sort them, and test whether a score exists.
06. [Original Ch 9] Draw a variable and a pointer to it.
07. [Original Ch 9] Write a safe function that accepts `int*` and handles `nullptr`.
08. [Original Ch 9] Describe the difference between `*p` and `&x`.
09. [Original Ch 9] Explain why pointer arithmetic needs bounds knowledge.
10. [Original Ch 9] EXTENSION: Create a two-number statistics function that returns the mean and writes min/max through validated pointer arguments.

4.12 Interactive quiz & video lessons

Interactive quiz — 12 auto-marked questions, instant feedback, explanations and chapter score.

[OPEN CHAPTER QUIZ ↗](#) • Opens this chapter's quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • **Verified public video lessons (external / internet needed)**

[Pointers and address basics \(freeCodeCamp, 2:59:11\) ↗](#)

[Searching and sorting: related C++ algorithm lessons \(The Chernov playlist\) ↗](#)

 [BACK TO CONTENTS](#)

CHAPTER 05 | Strings, Characters & Structured Records

Process text and group related values in records.

ORIGINAL HANDOUT COVERAGE • Chapter 10: Characters, C-Strings, and the string Class, Chapter 11: Structured Data

LEARNING ROUTE / Read → Trace → Run → Check

5.1 Character classification

`<cctype>` contains functions such as `isalpha`, `isdigit`, `toupper`, and `tolower`. For portable handling with potentially negative char values, pass an unsigned-char conversion to classification functions. These byte-oriented functions are not full Unicode case-conversion tools.

5.2 C-strings and termination

A C-string is an array of characters ending in a null character. Functions such as `std::strlen` inspect the sequence until this terminator. Destination buffers must be large enough for characters plus the null terminator; unsafe copying can overwrite memory.

5.3 Conversion and building helpers

Text that represents a number can be converted using suitable standard library functions, with error handling. A helper should state its assumptions: permitted characters, maximum input size, and what to do if conversion fails. Avoid writing a custom parser merely because the source contains one.

5.4 The `std::string` class

`std::string` supports `size()`, `empty()`, `find()`, `substr()`, `insert()`, and concatenation. Unlike a C-string, it manages its own storage. Its size counts char units; in UTF-8, this is not necessarily the number of visible characters. Access to the first element requires checking that the string is nonempty.

WATCH OUT If `find` returns `npos`, using that value as an ordinary valid index is a bug.

5.5 Group related fields

A struct defines a new type containing related named members. An object of this type holds one complete record such as student name, identifier, and mark. A structure declaration ends with a semicolon and the member variables are accessed using the dot operator.

5.6 Initialization and collections

Brace initialization supplies starting member values in declaration order. An array or vector of structures stores multiple records, and a traversal can calculate statistics or print a report. Grouping related information reduces the risk of misaligning separate parallel arrays.

5.7 Passing and accessing records

A structure can be passed by value, by reference, or through a pointer. Use const reference for a read-only record when a copy is unnecessary. Pointer-to-structure member access uses `->`, equivalent to dereferencing the pointer and then applying the dot operator.

5.8 Enums and unions

An enum defines a named set of alternatives, useful for discrete states. A union lets members share storage; using it safely requires tracking which member currently contains a valid value. This introductory handout emphasizes structs and enums; tagged unions or `std::variant` are safer for many modern applications.

WATCH OUT Accessing a member via a type name instead of an actual structure object is incorrect.

5.9 Key terms in this chapter

char; isalpha; isdigit; toupper; null terminator; C-string; string; find • struct; member; dot operator; ->; enum; array of structs; record

5.10 Guided C++ examples and answer traces

MINI-EXAMPLE (original Chapter 10)

WORKED MINI-EXAMPLE | Small example: check a search result

```
std::string word = "compiler";
auto at = word.find("pile");
std::cout << at << "\n";
```

How it works / expected result: Output is 3. Check against `std::string::npos` before using a position as an index.

Original Chapter 10: Find a word in a line of text

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain `main()`.

```
#include <iostream>
#include <string>
int main() {
    std::string sentence = "C++ helps solve problems";
    std::string target = "solve";
    std::size_t position = sentence.find(target);
    if (position == std::string::npos)
        std::cout << "Not found\n";
    else
        std::cout << "Found at index " << position << '\n';
    return 0;
}
```

MINI-EXAMPLE (original Chapter 11)

WORKED MINI-EXAMPLE | Small example: independent struct values

```
Student a{"Amina", 70};
Student b = a;
b.mark = 90;
```

How it works / expected result: The first Student still has mark 70. A normal struct copy creates independent member values.

Original Chapter 11: Struct-based student report

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain `main()`.

```
#include <iostream>
#include <string>
struct Student {
    std::string name;
    int mark;
};
int main() {
    Student student{"Lina", 88};
    std::cout << student.name << ": " << student.mark;
    std::cout << '\n';
    return 0;
}
```

INPUT / OUTPUT REFERENCE • Original Chapter 10

Sample input / starting data: sentence = C++ helps solve problems; target = solve

Expected output: Found at index 10

ANSWER TRACE • Original Chapter 10

1. `find` returns the first index of the matching text.
2. Compare the result with `std::string::npos` for no match.
3. Position 10 identifies the first character of `solve`.
4. Change target to an absent word and verify the other branch.

INPUT / OUTPUT REFERENCE • Original Chapter 11

Sample input / starting data: Student {Lina, 88}

Expected output: Lina: 88

ANSWER TRACE • Original Chapter 11

1. The struct declaration creates a type, not a specific record.
2. Brace initialization creates one Student object.
3. The dot operator reads both members of that object.
4. Extend the type with an id field and update its initialization.

5.11 Practice and check your understanding

01. [Original Ch 10] Compare `cin >> text` with `getline(cin, text)`.
02. [Original Ch 10] Count digits in an ASCII string using `isdigit` safely.
03. [Original Ch 10] Explain the null terminator in a C-string.
04. [Original Ch 10] Use `find` to locate a substring or report absence.

05. [Original Ch 10] EXTENSION: Write a text search utility that counts exact substring occurrences without reading past the end of the string.
06. [Original Ch 11] Define a struct for a book with title and price.
07. [Original Ch 11] Create and print an array of three Student records.
08. [Original Ch 11] Write a function that accepts const Student&.
09. [Original Ch 11] Demonstrate the -> operator using a valid Student*.
10. [Original Ch 11] EXTENSION: Build a library catalog that stores three book records and finds the most expensive book.

5.12 Interactive quiz & video lessons

Interactive quiz — 12 auto-marked questions, instant feedback, explanations and chapter score.

[OPEN CHAPTER QUIZ ↗](#) • Opens this chapter's quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • **Verified public video lessons (external / internet needed)**

[Strings and string operations \(freeCodeCamp, 39:15\) ↗](#)

[Classes versus structs and string representation \(The Cherno playlist\) ↗](#)

 [BACK TO CONTENTS](#)

CHAPTER 06 | File Processing & Class Fundamentals

Reliable file I/O, records, classes and encapsulation.

ORIGINAL HANDOUT COVERAGE • Chapter 12: Advanced File Operations, Chapter 13: Introduction to Classes

LEARNING ROUTE / Read → Trace → Run → Check

6.1 File streams and modes

<fstream> provides input, output, and bidirectional streams. Opening a file specifies a path and may include flags such as `std::ios::app` or `std::ios::binary`. Streams have state flags that indicate open/read/write failures. File formatting can use familiar `iostream` manipulators.

6.2 Safe read and write

Check whether the stream opened before using it. A loop such as `while (input >> number)` reads until extraction stops; distinguish normal end-of-file from malformed data when required. Pass a file stream to a helper by reference because streams are not copyable.

6.3 Binary and record data

Binary I/O reads and writes bytes rather than formatted decimal text. A simple fixed-layout record can be serialized with care, but raw bytes of a type containing `std::string`, pointers, or other dynamic members are not a portable serialization format. Document data format and error behavior.

6.4 Random access

`seekg` adjusts an input read position and `seekp` adjusts an output write position. `tellg` and `tellp` report positions when the operation succeeds. Random access is useful for fixed-size records, but offsets and record formats must be validated to avoid incorrect reads.

WATCH OUT Do not write raw bytes of a struct containing `std::string` and expect portable, reusable records.

6.5 Model data and behavior

An object combines state with actions that operate on that state. A class describes the common structure; individual objects have their own member values. This is different from a procedural function that manipulates unrelated global values.

6.6 Encapsulation

Declare internal data private so outside code cannot freely create invalid states. Expose a small public interface such as `deposit`, `balance`, and `withdraw`. The class should preserve its own invariants, for example that an account balance cannot become negative if that rule is part of the design.

6.7 Constructors and lifetime

A constructor establishes an initial state when an object is created. A destructor runs when an object is destroyed and can release resources owned by the object. Prefer ordinary standard-library members to manual resource management when possible.

WATCH OUT Public data members allow client code to bypass validation rules.

6.8 Key terms in this chapter

`ifstream`; `ofstream`; `fstream`; open mode; binary; read; write; `seekg`; `seekp` • class; object; private; public; constructor; destructor; encapsulation

6.9 Guided C++ examples and answer traces

MINI-EXAMPLE (original Chapter 12)

WORKED MINI-EXAMPLE | Small example: file failure branch

```
std::ifstream in("marks.txt");
if (!in) { std::cerr << "File missing\n"; }
```

How it works / expected result: An open check prevents treating a missing file as an empty data set. Supply `marks.txt` before running.

Original Chapter 12: Read a validated sequence from a file

The following is a complete, original C++17 example. Compile it as its own `.cpp` file; do not combine several examples that each contain `main()`.

```
#include <fstream>
#include <iostream>
int main() {
    std::ifstream file("marks.txt");
    if (!file) {
```

```

        std::cerr << "Cannot open marks.txt\n";
        return 1;
    }
    int mark = 0, total = 0, count = 0;
    while (file >> mark) { total += mark; ++count; }
    if (!file.eof()) {
        std::cerr << "Invalid file data\n";
        return 1;
    }
    std::cout << "Count = " << count
              << ", Total = " << total << '\n';
    return 0;
}

```

MINI-EXAMPLE (original Chapter 13)

WORKED MINI-EXAMPLE | Small example: encapsulation

```

Account wallet(50.0);
wallet.deposit(10.0);
std::cout << wallet.balance();

```

How it works / expected result: Output is 60. The caller cannot directly change the private `balance_` member.

Original Chapter 13: A bank account with controlled updates

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain `main()`.

```

#include <iostream>
class Account {
    double balance_ = 0.0;
public:
    explicit Account(double initial) : balance_(initial) {}
    void deposit(double amount) {
        if (amount > 0.0) balance_ += amount;
    }
    double balance() const { return balance_; }
};
int main() {
    Account savings(100.0);
    savings.deposit(25.0);
    std::cout << "Balance = " << savings.balance() << '\n';
    return 0;
}

```

INPUT / OUTPUT REFERENCE • Original Chapter 12

Sample input / starting data: marks.txt contains three integer lines: 70, 80, 90

Expected output: Count = 3, Total = 240

ANSWER TRACE • Original Chapter 12

1. Open marks.txt in the program working directory.
2. Reject an open failure before reading.
3. The extraction loop adds each valid integer to the total.
4. The final check differentiates a malformed token from ordinary end-of-file.

File requirement: create marks.txt with the integers 70, 80, and 90 on separate lines before running this program. The file must be in the executable's working directory.

INPUT / OUTPUT REFERENCE • Original Chapter 13

Sample input / starting data: initial amount 100; deposit 25

Expected output: Balance = 125

ANSWER TRACE • Original Chapter 13

1. The constructor initializes the private member.
2. Only deposit and balance are exposed as public operations.
3. A positive deposit changes state; a nonpositive one is ignored.
4. The const accessor reads the current state without changing the object.

6.10 Practice and check your understanding

01. [Original Ch 12] Create a file containing five integer values.
02. [Original Ch 12] Read integers until extraction fails and calculate a total.
03. [Original Ch 12] Describe the difference between text and binary formats.
04. [Original Ch 12] Explain why a stream is passed by reference.

05. [Original Ch 12] EXTENSION: Design a file-based grade register that stores validated text records and produces a summary report.
06. [Original Ch 13] Define a class for a rectangle with private dimensions.
07. [Original Ch 13] Add a constructor that initializes both dimensions.
08. [Original Ch 13] Write a const member function that returns area.
09. [Original Ch 13] Explain the difference between class and object.
10. [Original Ch 13] EXTENSION: Create a Temperature class with a validated setter and read-only Celsius/Fahrenheit accessors.

6.11 Interactive quiz & video lessons

Interactive quiz — 12 auto-marked questions, instant feedback, explanations and chapter score.

[OPEN CHAPTER QUIZ ↗](#) • Opens this chapter's quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • [Verified public video lessons \(external / internet needed\)](#)

[Read/write text and binary files \(EvoluNoob\) ↗](#)

[Classes, constructors and methods \(freeCodeCamp, 3:13:26\) ↗](#)

 [BACK TO CONTENTS](#)

CHAPTER 07 | Object-Oriented Design & Polymorphism

Object lifetime, copy semantics, inheritance and virtual dispatch.

ORIGINAL HANDOUT COVERAGE • Chapter 14: More About Classes, Chapter 15: Inheritance, Polymorphism, and Virtual Functions

LEARNING ROUTE / Read → Trace → Run → Check

7.1 Shared versus instance state

Each object normally owns a separate copy of nonstatic data members. A static data member belongs to the class as a whole and can track shared information such as the count of active objects. Avoid using static state when a local value would be clearer.

7.2 Copy behavior

Copy initialization creates a new object from an existing one; assignment changes an already existing object. Standard-library members such as `std::string` and `std::vector` already manage copies appropriately. A class that manually owns memory requires careful copy/move design to avoid double deletion.

7.3 Operator overloading

A class may define an operator to make a domain operation readable. The operator should have an unsurprising meaning: adding two displacement vectors can naturally use `+`, but using `+` to delete data would be misleading. Operator functions must obey permitted signatures and language restrictions.

WATCH OUT A shallow copy of an owning raw pointer can make two objects try to delete the same allocation.

7.4 Inheritance relationships

A derived class can reuse and extend an existing base-class interface. This models a genuine “is-a” relationship, not simply two classes sharing some fields. Public inheritance preserves an accessible base interface in the derived type.

7.5 Construction and overrides

Base subobjects are initialized before the derived-class body. A derived class can provide new behavior for a virtual base function; using `override` asks the compiler to verify that an actual override occurs. Prefer clear access rules rather than exposing all data as protected.

7.6 Virtual dispatch

When a virtual function is invoked through a base-class reference or pointer, the override associated with the actual object is selected. A polymorphic base class should usually have a virtual destructor so deleting a derived object through a base pointer destroys it correctly.

WATCH OUT An ordinary nonvirtual base function does not provide runtime virtual dispatch.

7.7 Key terms in this chapter

static; copy constructor; assignment; operator overloading; aggregation; composition • base class; derived class; inheritance; override; virtual; abstract; polymorphism

7.8 Guided C++ examples and answer traces

MINI-EXAMPLE (original Chapter 14)

WORKED MINI-EXAMPLE | Small example: independent copies

```
Label a("One");
Label b = a;
b.set("Two");
```

How it works / expected result: `a.text()` stays One while `b.text()` is Two. Copy semantics matter when classes own resources.

Original Chapter 14: Copying standard-library-owned members

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain `main()`.

```
#include <iostream>
#include <string>
class Label {
    std::string text_;
public:
    explicit Label(std::string text) : text_(text) {}
    void set(std::string text) { text_ = text; }
    std::string text() const { return text_; }
};
int main() {
```

```

    Label first("Original");
    Label second = first;
    second.set("Copy");
    std::cout << first.text() << " / " << second.text() << '\n';
    return 0;
}

```

MINI-EXAMPLE (original Chapter 15)

WORKED MINI-EXAMPLE | Small example: dispatch via reference

```

Circle circle;
const Shape& shape = circle;
shape.describe();

```

How it works / expected result: Output is A circle. Virtual dispatch occurs through a base reference or pointer.

Original Chapter 15: Virtual display through a base reference

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```

#include <iostream>
class Shape {
public:
    virtual ~Shape() = default;
    virtual void describe() const = 0;
};
class Circle : public Shape {
public:
    void describe() const override {
        std::cout << "A circle\n";
    }
};
int main() {
    Circle circle;
    const Shape& item = circle;
    item.describe();
    return 0;
}

```

INPUT / OUTPUT REFERENCE • Original Chapter 14

Sample input / starting data: copy Label containing Original, change the copy

Expected output: Original / Copy

ANSWER TRACE • Original Chapter 14

1. Copy construction creates second from first.
2. std::string copies its contents correctly.
3. Changing second does not replace the text in first.
4. For a class managing raw dynamically allocated memory, explain why the default copy might not be sufficient.

INPUT / OUTPUT REFERENCE • Original Chapter 15

Sample input / starting data: a Circle object accessed as a Shape reference

Expected output: A circle

ANSWER TRACE • Original Chapter 15

1. Shape declares the required interface.
2. Circle supplies an override using exactly the expected signature.
3. The base reference refers to the existing Circle object.
4. The virtual call selects the derived override.

7.9 Practice and check your understanding

01. [Original Ch 14] Distinguish a static counter from an instance value.
02. [Original Ch 14] Show copy construction and assignment on a simple class.
03. [Original Ch 14] Overload + for a two-coordinate vector.
04. [Original Ch 14] Explain why operator meanings should be natural.
05. [Original Ch 14] EXTENSION: Implement a Point class with coordinate getters, a constructor, and addition of two points.
06. [Original Ch 15] Explain an is-a relationship with a meaningful example.
07. [Original Ch 15] Add a second derived shape to the sample.
08. [Original Ch 15] Explain what override helps the compiler detect.
09. [Original Ch 15] Create an abstract PaymentMethod interface.
10. [Original Ch 15] EXTENSION: Design a small hierarchy of notifications with a virtual send() operation.

7.10 Interactive quiz & video lessons

Interactive quiz — 12 auto-marked questions, instant feedback, explanations and chapter score.

[OPEN CHAPTER QUIZ ↗](#) • Opens this chapter's quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • **Verified public video lessons (external / internet needed)**

[Inheritance introduction \(freeCodeCamp, 3:54:04\) ↗](#)

[Virtual functions, copying and class lifetime \(The Chernobyl playlist\) ↗](#)

 [BACK TO CONTENTS](#)

CHAPTER 08 | Generic Programming & Linked Lists

Exceptions, templates, STL containers and linked data.

ORIGINAL HANDOUT COVERAGE • Chapter 16: Exceptions, Templates, and the STL, Chapter 17: Linked Lists

LEARNING ROUTE / Read → Trace → Run → Check

8.1 Exception handling

An exception reports that an operation could not complete normally. Code in a try block may throw; an appropriate catch can handle the failure. Use exceptions for exceptional conditions, and avoid silently swallowing errors. Standard library operations such as `vector::at` may throw on invalid indexing.

8.2 Templates

A function template describes an operation for a family of compatible types. Template argument deduction lets a compiler instantiate code for types such as `int` and `double`. A class template similarly defines a family of classes, such as `std::vector<int>` and `std::vector<std::string>`.

8.3 Containers and iterators

The standard library offers sequence containers and associative containers, plus iterator-based algorithms. `begin` and `end` specify a range; `end` marks one past its final element. Choose containers according to required access, insertion, and traversal behavior rather than convenience alone.

WATCH OUT Catch exceptions at a level that can respond meaningfully; an empty catch can hide a serious failure.

8.4 Nodes and links

A singly linked list consists of nodes with a value and a link to the next node. Its head identifies the first node; a null link marks the end. Unlike array elements, nodes do not need to occupy one contiguous block of memory.

8.5 Traversal and insertion

A traversal starts at the head and follows next pointers until the end. Inserting at the front changes the new node's next link to the old head, then updates head. Deletion must preserve access to the remaining nodes before releasing the removed node.

8.6 Ownership and variants

Manual lists require a deliberate memory-ownership strategy. A singly linked list supports movement in one direction; a doubly linked list also includes a previous link. A circular list connects its last node back to an earlier node and needs a careful stopping condition.

WATCH OUT Overwriting a link before preserving the rest of the chain can make nodes unreachable.

8.7 Key terms in this chapter

try; catch; throw; template; STL; iterator; vector; algorithm • node; link; head; next; traversal; insertion; deletion; `std::list`

8.8 Guided C++ examples and answer traces

MINI-EXAMPLE (original Chapter 16)

WORKED MINI-EXAMPLE | Small example: exception-safe access

```
std::vector<int> v{2, 4};
std::cout << v.at(0);
```

How it works / expected result: Output is 2. `v.at(2)` would throw `std::out_of_range`; `v[2]` does not check bounds.

Original Chapter 16: A template and a checked collection

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain `main()`.

```
#include <iostream>
#include <vector>
#include <stdexcept>
template <typename T>
T larger(T a, T b) { return (a < b) ? b : a; }
int main() {
    std::vector<int> values = {5, 10};
    try {
        std::cout << "Larger = " << larger(5, 8) << '\n';
        std::cout << "Value = " << values.at(1) << '\n';
    } catch (const std::out_of_range& error) {
        std::cerr << error.what() << '\n';
    }
```

```

        return 1;
    }
    return 0;
}

```

MINI-EXAMPLE (original Chapter 17)

WORKED MINI-EXAMPLE | Small example: remove from a list

```

std::list<int> ids{2, 4, 6};
ids.pop_front();
std::cout << ids.front();

```

How it works / expected result: Output is 4. Check empty() before calling front() or pop_front() on a potentially empty list.

Original Chapter 17: Standard linked-list operations

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```

#include <iostream>
#include <list>
int main() {
    std::list<int> numbers = {3, 5, 7};
    numbers.push_front(1);
    numbers.push_back(9);
    for (int value : numbers)
        std::cout << value << ' ';
    std::cout << '\n';
    return 0;
}

```

INPUT / OUTPUT REFERENCE • Original Chapter 16

Sample input / starting data: integers 5 and 8; vector element at index 1

Expected output: Larger = 8 | Value = 10

ANSWER TRACE • Original Chapter 16

1. larger<int> is instantiated using integer arguments.
2. The conditional expression selects 8.
3. at(1) safely reads the second vector element.
4. If the index is invalid, an out_of_range exception can be caught.

INPUT / OUTPUT REFERENCE • Original Chapter 17

Sample input / starting data: list {3,5,7}, insert 1 at front and 9 at back

Expected output: 1 3 5 7 9

ANSWER TRACE • Original Chapter 17

1. The initial list contains three nodes.
2. push_front adds an element before the old head.
3. push_back adds an element after the old tail.
4. Range traversal prints the five elements in order.

8.9 Practice and check your understanding

01. [Original Ch 16] Write a function template to compute a minimum.
02. [Original Ch 16] Describe the difference between a template and overloads.
03. [Original Ch 16] Catch an out_of_range exception from vector::at.
04. [Original Ch 16] Trace an iterator range [begin, end).
05. [Original Ch 16] EXTENSION: Build a template-based numeric summary that finds a minimum and maximum over a nonempty vector.
06. [Original Ch 17] Draw a three-node singly linked list with arrows.
07. [Original Ch 17] Explain how front insertion changes the head.
08. [Original Ch 17] Trace traversal from head to null.
09. [Original Ch 17] Identify the risk when deleting a node before saving its next pointer.
10. [Original Ch 17] EXTENSION: Create a linked-list inventory demo that appends items, prints them, and removes one selected value.

8.10 Interactive quiz & video lessons

Interactive quiz — 12 auto-marked questions, instant feedback, explanations and chapter score.

[OPEN CHAPTER QUIZ ↗](#) • Opens this chapter's quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • **Verified public video lessons (external / internet needed)**

[Templates, standard containers and linked-list foundations \(The Chernobyl playlist\) ↗](#)

[Linked lists in C/C++ \(freeCodeCamp, 19:40\) ↗](#)

 [BACK TO CONTENTS](#)

CHAPTER 09 | Stacks, Queues, Recursion & Trees

Abstract data types, recursive thinking and binary search trees.

ORIGINAL HANDOUT COVERAGE • Chapter 18: Stacks and Queues, Chapter 19: Recursion, Chapter 20: Binary Trees

LEARNING ROUTE / Read → Trace → Run → Check

9.1 Stack behavior

A stack supports insertion and removal at the same end. push adds an item and pop removes the most recently added item, giving last-in-first-out order. Before calling top or pop, check that the stack is not empty.

9.2 Queue behavior

A queue adds at the rear and removes from the front. This first-in-first-out behavior models waiting lines and work processing. Check empty before using front or pop. Standard queue is a container adaptor, so it intentionally does not expose general indexing.

9.3 Representations

A stack can be built using an array, vector, or linked nodes. A queue can be built using linked nodes or a circular array with front/rear indices. For learning, trace which end changes and which invariants must stay true after each operation.

WATCH OUT Calling top, front, or pop on an empty adaptor is invalid; test empty first.

9.4 Self-referential solutions

A recursive function solves a task by calling itself on a smaller instance. Every correct recursive design needs a base case that stops the process and a recursive step that moves toward that case. The call stack stores each invocation's local state until it returns.

9.5 Proving termination

Write down a measure that gets smaller on every recursive call, such as n decreasing to zero or an interval width shrinking. A missing base case or a step that does not make progress can exhaust the call stack.

9.6 Classic recursive examples

Factorial, Euclid's greatest common divisor, binary search, and tree traversal can be expressed recursively. The recurrence should describe the problem correctly: $\text{gcd}(a,b) = \text{gcd}(b,a\%b)$ when b is nonzero, with $\text{gcd}(a,0)$ determined by a under appropriate nonnegative assumptions.

WATCH OUT A recursive call that does not approach a base case can repeat until stack exhaustion.

9.7 Tree structure

A binary tree is a linked hierarchy where a node has at most two children. The root has no parent; leaf nodes have no children. Trees model hierarchical data and recursive subdivisions. Height measures longest root-to-leaf path using a chosen convention.

9.8 Binary search trees

A binary search tree arranges keys so the left subtree holds smaller keys and the right subtree holds larger keys under a consistent duplicate policy. Searching follows one branch after each comparison. An unbalanced tree can become a chain, making search performance much worse than for a balanced tree.

9.9 Insertion and traversal

Insertion follows comparisons until it reaches an empty child position. Inorder traversal visits left, node, right and therefore prints keys in ascending order for a BST with suitable ordering. Preorder and postorder are also useful for copying or processing hierarchical structures.

WATCH OUT A binary search tree is not automatically balanced; insertion order can create a degenerate chain.

9.10 Key terms in this chapter

stack; push; pop; top; queue; front; back; LIFO; FIFO • recursion; base case; recursive case; call stack; gcd; binary search • root; node; child; leaf; BST; inorder; preorder; postorder

9.11 Guided C++ examples and answer traces

MINI-EXAMPLE (original Chapter 18)

WORKED MINI-EXAMPLE | Small example: removal order

```
std::stack<int> s;
s.push(4); s.push(9);
std::cout << s.top();
```

How it works / expected result: Output is 9. After pop(), the top would be 4; only inspect top() when not empty.

Original Chapter 18: One stack, one queue

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```
#include <iostream>
#include <queue>
#include <stack>
int main() {
    std::stack<int> lastIn;
    std::queue<int> firstIn;
    for (int value : {10, 20, 30}) {
        lastIn.push(value);
        firstIn.push(value);
    }
    std::cout << "Stack top = " << lastIn.top() << '\n';
    std::cout << "Queue front = " << firstIn.front() << '\n';
    return 0;
}
```

MINI-EXAMPLE (original Chapter 19)

WORKED MINI-EXAMPLE | Small example: terminating recursion

```
int sumTo(int n) {
    if (n <= 0) return 0;
    return n + sumTo(n - 1);
}
```

How it works / expected result: sumTo(3) is 6. Every recursive path must approach a base case.

Original Chapter 19: Euclidean greatest common divisor

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```
#include <iostream>
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
int main() {
    int a = 24, b = 18;
    std::cout << "GCD = " << gcd(a, b) << '\n';
    return 0;
}
```

MINI-EXAMPLE (original Chapter 20)

WORKED MINI-EXAMPLE | Small example: inorder reasoning

Tree: root 5, left 2, right 8.
Visit left → root → right.
Result: 2 5 8

How it works / expected result: A binary search tree is not automatically balanced. Sorted insertion can produce a chain.

Original Chapter 20: A small binary search tree

The following is a complete, original C++17 example. Compile it as its own .cpp file; do not combine several examples that each contain main().

```
#include <iostream>
#include <memory>
struct Node {
    int value;
    std::unique_ptr<Node> left, right;
    explicit Node(int v) : value(v) {}
};
void insert(std::unique_ptr<Node>& node, int value) {
    if (!node) { node = std::make_unique<Node>(value); return; }
    if (value < node->value) insert(node->left, value);
    else if (value > node->value) insert(node->right, value);
}
void inorder(const std::unique_ptr<Node>& node) {
    if (!node) return;
    inorder(node->left);
    std::cout << node->value << ' ';
    inorder(node->right);
}
int main() {
    std::unique_ptr<Node> root;
    for (int v : {5, 2, 8, 1}) insert(root, v);
    inorder(root);
    std::cout << '\n';
    return 0;
}
```

INPUT / OUTPUT REFERENCE • Original Chapter 18

Sample input / starting data: push 10, 20, 30 into both containers

Expected output: Stack top = 30 | Queue front = 10

ANSWER TRACE • Original Chapter 18

1. Both containers receive values in the same insertion order.
2. The stack exposes the newest value at its top.
3. The queue exposes the oldest value at its front.
4. Predict results after one pop from each container.

INPUT / OUTPUT REFERENCE • Original Chapter 19

Sample input / starting data: a = 24, b = 18

Expected output: GCD = 6

ANSWER TRACE • Original Chapter 19

1. gcd(24,18) calls gcd(18,6).
2. gcd(18,6) calls gcd(6,0).
3. The base case returns 6.
4. Each suspended call returns that result to its caller.

INPUT / OUTPUT REFERENCE • Original Chapter 20

Sample input / starting data: insert 5, 2, 8, 1 into an empty BST

Expected output: 1 2 5 8

ANSWER TRACE • Original Chapter 20

1. The first value becomes root.
2. 2 enters the left subtree and 8 the right.
3. 1 becomes the left child under 2.
4. Inorder traversal prints the keys in sorted order.

The helper functions use `std::unique_ptr` to retain ownership of every allocated node; inorder traversal displays the values in sorted order.

9.12 Practice and check your understanding

01. [Original Ch 18] Trace push(2), push(4), pop(), push(6) on a stack.
02. [Original Ch 18] Trace enqueue(A), enqueue(B), dequeue() on a queue.
03. [Original Ch 18] Explain LIFO and FIFO using practical examples.
04. [Original Ch 18] Describe an empty-container guard.
05. [Original Ch 18] EXTENSION: Create a simple task processor that enqueues jobs and handles them in arrival order.
06. [Original Ch 19] Define a base case for recursive factorial.
07. [Original Ch 19] Trace the recursive calls made by gcd(20,12).
08. [Original Ch 19] Explain what the call stack stores.
09. [Original Ch 19] Compare a recursive and iterative countdown.
10. [Original Ch 19] EXTENSION: Implement recursive binary search on a sorted vector and compare its tested output with `std::binary_search`.
11. [Original Ch 20] Draw the BST produced by inserting 7, 3, 9, 1.
12. [Original Ch 20] Give inorder, preorder, and postorder traversals.
13. [Original Ch 20] Explain a BST ordering invariant.
14. [Original Ch 20] Describe why a chain-shaped tree has poor search behavior.
15. [Original Ch 20] EXTENSION: Build an integer BST with insertion, inorder display, and a search function.

9.13 Interactive quiz & video lessons

Interactive quiz — 18 auto-marked questions, instant feedback, explanations and chapter score.

[OPEN CHAPTER QUIZ ↗](#) • Opens this chapter's quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • Verified public video lessons (external / internet needed)

[Stacks and queues \(freeCodeCamp, 2:43:09\) ↗](#)

[Binary trees and BSTs \(freeCodeCamp, 5:10:48\) ↗](#)



[BACK TO CONTENTS](#)

CHAPTER 10 | Solved C++ Projects & Answered Examples

Three complete C++17 projects with specifications, explanations, expected output and tests.

10.1 Validated Menu Calculator

FOUNDATIONS • DECISIONS • INPUT VALIDATION

Problem statement. Read two numbers and an operation code (1 add, 2 subtract, 3 multiply, 4 divide). Reject malformed input, invalid menu choices and division by zero.

Solution plan

- Input: operation (1–4), two double values.
- Process: validate choice and input; dispatch with switch.
- Output: the operation result or a clear error message.

Complete, compilable C++17 solution

```
#include <iostream>

int main() {
    int choice = 0;
    double a = 0.0, b = 0.0;
    std::cout << "Operation (1:+, 2:-, 3:*, 4:/): ";
    if (!(std::cin >> choice) || choice < 1 || choice > 4) {
        std::cout << "Invalid operation\n";
        return 1;
    }
    std::cout << "Enter two numbers: ";
    if (!(std::cin >> a >> b)) {
        std::cout << "Invalid numbers\n";
        return 1;
    }
    double answer = 0.0;
    switch (choice) {
        case 1: answer = a + b; break;
        case 2: answer = a - b; break;
        case 3: answer = a * b; break;
        case 4:
            if (b == 0.0) {
                std::cout << "Division by zero is not allowed\n";
                return 1;
            }
            answer = a / b;
            break;
    }
    std::cout << "Result = " << answer << '\n';
    return 0;
}
```

Worked answer and expected result

Operation (1:+, 2:-, 3:*, 4:/): Enter two numbers: Result = 4

HOW IT WORKS / The first validation protects the switch; the second protects operands. The division branch checks the denominator before computing a result.

Test your solution

1. Input 1; 8 2 → Result = 10
2. Input 4; 8 0 → Division by zero is not allowed
3. Input 9 → Invalid operation

Source file: project_solutions/Project_1.cpp (included in ZIP package).

10.2 Student Record Analyzer

FUNCTIONS • STRUCTURES • VECTORS • SORTING

Problem statement. Read 1–100 student records (one-word names, integer marks 0–100), compute the class mean and pass count, then sort students from highest mark to lowest. Preserve names with their marks.

Solution plan

- Input: count, followed by name and mark pairs.
- Process: validate count and every mark; store complete Student records; accumulate and sort.

- Output: mean, pass count, highest record and ordered list.

Complete, compilable C++17 solution

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

struct Student {
    std::string name;
    int mark;
};

int main() {
    int count = 0;
    if (!(std::cin >> count) || count < 1 || count > 100) {
        std::cout << "Invalid student count\n";
        return 1;
    }
    std::vector<Student> students;
    int total = 0, passing = 0;
    for (int i = 0; i < count; ++i) {
        Student s{};
        if (!(std::cin >> s.name >> s.mark) ||
            s.mark < 0 || s.mark > 100) {
            std::cout << "Invalid student record\n";
            return 1;
        }
        total += s.mark;
        if (s.mark >= 50) ++passing;
        students.push_back(s);
    }
    std::sort(students.begin(), students.end(),
        [](const Student& a, const Student& b) {
            if (a.mark != b.mark) return a.mark > b.mark;
            return a.name < b.name;
        });
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Mean = " << double(total) / count << '\n';
    std::cout << "Pass count = " << passing << '\n';
    std::cout << "Highest = " << students.front().name << ' '
        << students.front().mark << '\n';
    std::cout << "Ranking:\n";
    for (const Student& s : students)
        std::cout << s.name << ' ' << s.mark << '\n';
    return 0;
}
```

Worked answer and expected result

```
Mean = 71.67
Pass count = 3
Highest = Mona 90
Ranking:
Mona 90
Sara 75
Ali 50
```

HOW IT WORKS / A struct keeps each name paired with the correct mark. The program refuses invalid data instead of averaging it. Sorting rearranges entire Student objects; count is known to be nonzero.

Test your solution

1. Input 1; Ali 50 → Mean = 50.00; pass count = 1
2. Input 2; Mona 90; Ali 49 → Mean = 69.50; pass count = 1
3. Input 1; Ali 101 → Invalid student record

Source file: project_solutions/Project_2.cpp (included in ZIP package).

10.3 Searchable Binary Search Tree

POINTERS • UNIQUE_PTR • RECURSION • BINARY TREES

Problem statement. Read integer keys, insert them into a binary search tree, show the sorted inorder traversal, then report whether a query exists.

Solution plan

- Input: number of keys, keys and target.

- Process: insert smaller keys left/larger keys right; ignore duplicates; search recursively.
- Output: inorder keys and FOUND/NOT FOUND.

Complete, compilable C++17 solution

```
#include <iostream>
#include <memory>

struct Node {
    int key;
    std::unique_ptr<Node> left, right;
    explicit Node(int value) : key(value) {}
};

void insert(std::unique_ptr<Node>& node, int key) {
    if (!node) { node = std::make_unique<Node>(key); return; }
    if (key < node->key) insert(node->left, key);
    else if (key > node->key) insert(node->right, key);
}

bool contains(const Node* node, int target) {
    if (!node) return false;
    if (target == node->key) return true;
    return target < node->key
        ? contains(node->left.get(), target)
        : contains(node->right.get(), target);
}

void inorder(const Node* node) {
    if (!node) return;
    inorder(node->left.get());
    std::cout << ' ' << node->key;
    inorder(node->right.get());
}

int main() {
    int n = 0;
    if (!(std::cin >> n) || n < 1 || n > 100) {
        std::cout << "Invalid count\n";
        return 1;
    }
    std::unique_ptr<Node> root;
    for (int i = 0; i < n; ++i) {
        int key = 0;
        if (!(std::cin >> key)) {
            std::cout << "Invalid key\n";
            return 1;
        }
        insert(root, key);
    }
    int target = 0;
    if (!(std::cin >> target)) {
        std::cout << "Invalid target\n";
        return 1;
    }
    std::cout << "Inorder:";
    inorder(root.get());
    std::cout << '\n';
    std::cout << (contains(root.get(), target) ? "FOUND" : "NOT FOUND") << '\n';
    return 0;
}
```

Worked answer and expected result

```
Inorder: 1 3 7 9
FOUND
```

HOW IT WORKS / The ordering invariant directs insert/search to one child at a time. Inorder visits left-root-right, giving ascending keys. `unique_ptr` automatically frees each owned subtree. A plain BST is not necessarily balanced.

Test your solution

1. Input 4; 7 3 9 1; target 4 → NOT FOUND
2. Input 4; 7 3 7 1; target 7 → Inorder: 1 3 7; FOUND
3. Input 0 → Invalid count

Source file: `project_solutions/Project_3.cpp` (included in ZIP package).

10.4 How to submit and test a project

Compile each .cpp file independently with C++17. Capture one normal run, one boundary case and one invalid-input case. State which requirement each test checks. Save your .cpp, three console outputs and a short README.

Academic note: these are worked teaching examples, not the institution's official project assessment rubric. The source handout's Appendix C describes a student record analysis project; the other two worked programs extend the handout's calculator and BST exercises.

10.5 Interactive project quiz & supporting videos

8 project-specific MCQs with immediate correct/incorrect feedback, explanations and an overall score.

[OPEN CHAPTER 10 PROJECT QUIZ ↗](#) • Opens the project quiz on the student website.

[ALL CHAPTER QUIZZES ↗](#)

WATCH & LEARN • Supporting video sections

[Calculator worked example \(freeCodeCamp, 1:05:32\) ↗](#)

[Linked structures and BST project concepts \(freeCodeCamp, 5:42:51\) ↗](#)

 [BACK TO CONTENTS](#)